

ToDo-Compose应用软件测试与性能评估报告

测试人员：刘晨旭、王子文
时间：2026.6.13

一、项目总览-软件介绍

ToDo-Composes是一个安卓代办清单软件

支持插入代办事项并撰写描述

支持按着优先级排序

支持通过搜索查找代办事项

支持滑动手势删除 + 撤销

深色/浅色主题切换

包含流畅的动画效果

简洁直观的操作逻辑

帮助用户理清规划自己要做什么事

技术栈: Kotlin + Jetpack Compose + Room +
DataStore + Hilt + Navigation Compose



一、项目总览-选题原因

5 月 14 日上午第一节课发布了选题确认通知，并要求于当天上午 9:30 前完成选题。由于时间较为紧张，我在短时间内借助 AI 工具筛选了若干 Android 开源应用作为候选项目。然而，在实际配置与运行过程中，大部分项目均因 Android Studio 版本、Gradle 依赖或环境兼容性问题导致编译失败。

在截止时间前，只有 ToDo-Compose 项目能够在调整部分依赖版本后成功完成编译与运行。因此，综合时间限制与项目可运行性因素，最终选择了该项目作为后续实验与测试分析对象。

一、项目总览-项目规模

工程量评估

工程规模： 小 (~30 个源文件, 不含测试)

复杂度： 低~中

架构模式： MVVM + Repository + DI

数据库： 1 张表 (ToDoTask), 5 个 DAO 查询

导航： 2 个目的地, 1 个共享 ViewModel

亮点： Compose 动画 (SwipeToDismiss, AnimatedVisibility, 旋转箭头), 协程 Flow, DataStore 偏好存储, Hilt 注入

一、项目总览-项目流程

MainActivity

- └─ SetupNavigation (NavHost)

- └─ ListScreen ← startDestination (list/{action})

- └─ 观察 SharedViewModel.allTask / searchedTasks / sortState

- └─ FAB(+) → navigate 到 TaskScreen(taskId=-1)

- └─ 点击任务 → navigate 到 TaskScreen(taskId=N)

- └─ Swipe 删除 → Action.DELETE → Snackbar(UNDO)

- └─ 搜索 → SharedViewModel.searchDatabase()

- └─ 排序 → DataStore 持久化 → 切换 Flow

- └─

- └─ TaskScreen (task/{taskId})

- └─ taskId=-1 → 新建模式 (NewTaskAppBar)

- └─ taskId=N → 编辑模式 (ExistingTaskAppBar)

- └─ 返回 → Action.ADD / UPDATE / DELETE / NO_ACTION

- └─ SharedViewModel.handleDatabaseActions() 调度

总指挥部: MainActivity

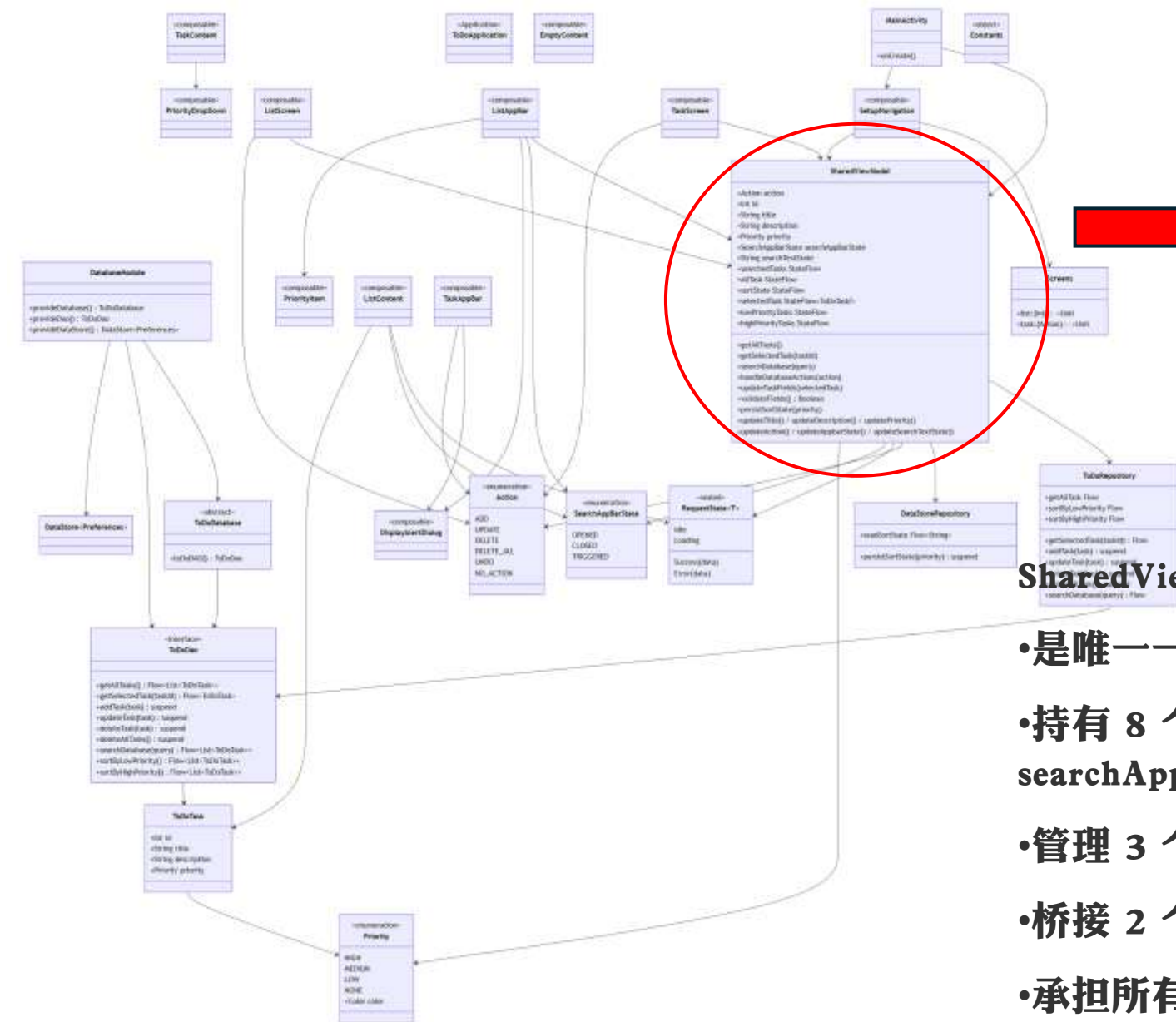
导航系统: SetupNavigation

第一页: 任务列表 (ListScreen)

第二页: 任务详情 (TaskScreen)

幕后大管家: SharedViewModel

一、项目总览-类的调用关系



重点类

SharedViewModel 是整个应用的中枢神经系统：

- 是唯一一个 ViewModel，所有 UI 组件全部依赖它
- 持有 8 个可观察状态（action, id, title, description, priority, searchAppBarState, searchTextState, selectedTask）
- 管理 3 个 StateFlow（allTask, searchedTasks, sortState）
- 桥接 2 个 Repository（ToDoRepository + DataStoreRepository）
- 承担所有 CRUD 调度、搜索、排序、字段校验、导航动作

二、单元测试-基于junit

最初阶段主要使用 JUnit 对枚举类与工具类进行基础单元测试，包括 Priority、Action、RequestState 与 SearchAppBarState 等核心状态类，共完成 37 个测试用例，全部通过。这一阶段被测试文件的行覆盖率均达到 100%，但由于项目主体仍以 Compose UI、Room 数据库与 ViewModel 为主，因此项目整体覆盖率仅约为 1%。

app

Sessions

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.ayse.todocompose.ui.screens.list		0%		0%	366	366	426	426	107	107	75	75
com.ayse.todocompose.ui.screens.task		0%		0%	231	231	187	187	54	54	40	40
com.ayse.todocompose.components		0%		0%	111	111	105	105	32	32	25	25
com.ayse.todocompose.ui.viewModel		0%		0%	66	66	143	143	55	55	16	16
com.ayse.todocompose.ui.theme		7%		0%	59	64	41	52	28	33	4	5
com.ayse.todocompose.navigation.destination		0%		0%	24	24	41	41	13	13	10	10
com.ayse.todocompose.data.repository		0%		0%	24	24	33	33	22	22	8	8
com.ayse.todocompose.navigation		0%		0%	12	12	23	23	9	9	7	7
com.ayse.todocompose		0%		0%	20	20	18	18	9	9	7	7
com.ayse.todocompose.data		0%		n/a	7	7	9	9	7	7	2	2
com.ayse.todocompose.di		0%		n/a	2	2	5	5	2	2	1	1
com.ayse.todocompose.util		100%		100%	0	12	0	14	0	9	0	7
com.ayse.todocompose.data.models		100%		n/a	0	3	0	5	0	3	0	1
Total	11,298 of 11,501	1%	1,164 of 1,170	0%	922	942	1,031	1,061	338	355	195	204

Created with JaCoCo 0.8.11.202310140853

二、单元测试-引入 MockK

随后，在原有基础上进一步引入 MockK，对 Repository 与 ViewModel 层进行了模拟测试。新增了 ToDoRepositoryTest、DataStoreRepositoryTest 与 SharedViewModelTest 等测试类，通过 mock 数据源、协程与 Flow 返回值，实现了对业务逻辑层的完整验证。此阶段测试总数扩展至 84 项，全部通过。多个核心类覆盖率显著提升，其中 ToDoRepository.kt 达到 100%，SharedViewModel.kt 达到 94.2%，项目整体测试覆盖率提升至约 11%。

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.ayse.todocompose.ui.screens.list		0%		0%	366	366	426	426	107	107	75	75
com.ayse.todocompose.ui.screens.task		0%		0%	231	231	187	187	54	54	40	40
com.ayse.todocompose.components		0%		0%	111	111	105	105	32	32	25	25
com.ayse.todocompose.ui.theme		7%		0%	59	64	41	52	28	33	4	5
com.ayse.todocompose.navigation.destination		0%		0%	24	24	41	41	13	13	10	10
com.ayse.todocompose.navigation		0%		0%	12	12	23	23	9	9	7	7
com.ayse.todocompose		0%		0%	20	20	18	18	9	9	7	7
com.ayse.todocompose.data.repository		73%		50%	4	24	4	32	3	22	2	8
com.ayse.todocompose.ui.viewModel		96%		100%	2	66	8	143	2	55	0	16
com.ayse.todocompose.di		0%		n/a	3	3	6	6	3	3	1	1
com.ayse.todocompose.data		94%		n/a	1	7	1	9	1	7	1	2
com.ayse.todocompose.util		100%		100%	0	12	0	14	0	9	0	7
com.ayse.todocompose.data.models		100%		n/a	0	3	0	5	0	3	0	1
Total	10,234 of 11,502	11%	1,144 of 1,170	2%	833	943	860	1,061	261	356	172	204

二、单元测试-对于重要的SharedViewModel

针对 SharedViewModel 编写的单元测试，主要使用了 JUnit、MockK 和 kotlin.coroutines.test 等工具，对 ViewModel 的状态管理和业务逻辑进行验证。测试中通过 mock 模拟了 ToDoRepository 和 DataStoreRepository，避免真正访问数据库，从而只关注 ViewModel 本身的逻辑是否正确。

测试内容主要包括三类：一是初始化状态测试，例如检查默认的 action、priority、搜索栏状态等是否正确；二是输入与状态更新测试，例如 updateTitle()、updateDescription()、validateFields() 等函数是否能够正确修改和校验数据；三是数据库行为测试，例如调用 handleDatabaseActions(Action.ADD) 后，是否真正触发了 addTask()，以及搜索、读取任务等功能是否能正确更新状态。

整体测试流程是：先构造 mock 数据和测试环境，然后调用 ViewModel 的方法，再通过 assertEquals、assertTrue、coVerify 等断言验证状态变化或 Repository 调用是否符合预期。

二、单元测试-对于重要的SharedViewModel

测试覆盖率达96%

com.ayse.todocompose.ui.viewModel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SharedViewModel		94%		100%	2	50	8	96	2	39	0	1
SharedViewModel.searchDatabase.new Function2().{...}		100%		n/a	0	1	0	3	0	1	0	1
SharedViewModel.updateTask.new Function2().{...}		100%		n/a	0	1	0	7	0	1	0	1
SharedViewModel.deleteTask.new Function2().{...}		100%		n/a	0	1	0	7	0	1	0	1
SharedViewModel.addTask.new Function2().{...}		100%		n/a	0	1	0	6	0	1	0	1
SharedViewModel.readSortState.new Function2().{...}		100%		n/a	0	1	0	4	0	1	0	1
SharedViewModel.getSelectedTask.new Function2().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.getAllTasks.new Function2().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.persistSortState.new Function2().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.readSortState.1.invokeSuspend..inlined.map.new Flow().{...}		100%		n/a	0	2	0	3	0	2	0	1
SharedViewModel.deleteAllTask.new Function2().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.readSortState.1.new FlowCollector().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.searchDatabase.1.new FlowCollector().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.getAllTasks.1.new FlowCollector().{...}		100%		n/a	0	1	0	2	0	1	0	1
SharedViewModel.readSortState.1.invokeSuspend..inlined.map.1.new FlowCollector().{...}		100%		n/a	0	1	0	1	0	1	0	1
SharedViewModel.getSelectedTask.1.new FlowCollector().{...}		100%		n/a	0	1	0	2	0	1	0	1
Total	33 of 876	96%	0 of 18	100%	2	66	8	143	2	55	0	16

二、单元测试-对于重要的SharedViewModel 举例

@Test

```
fun `handleDatabaseActions ADD calls addTask`() {  
    viewModel.updateTitle("t")  
    // 模拟用户输入任务标题 "t", 写入 ViewModel 的 title 状态  
    viewModel.updateDescription("d")  
    // 模拟用户输入任务描述 "d", 写入 ViewModel 的 description 状态  
    coEvery { mockRepo.addTask(any()) } returns Unit  
    // 使用 MockK 模拟 Repository 的 addTask 方法  
    // 表示: 只要调用 addTask, 就返回 Unit (不执行真实数据库操作)  
    // 这里用于隔离测试, 避免依赖真实数据层  
    viewModel.handleDatabaseActions(Action.ADD)  
    // 触发 ViewModel 的核心业务逻辑  
    // 等价于用户点击“新增任务”按钮  
    // ViewModel 会根据 Action.ADD 构建任务并调用 repository.addTask()  
    Thread.sleep(50)  
    coVerify { mockRepo.addTask(any()) }  
    // 验证 mockRepo 的 addTask 方法是否被调用过 这是关键断言: 确认“数  
    据写入动作确实发生了”  
    assertEquals(SearchAppBarState.CLOSED, viewModel.searchAppBarState)  
    // 验证 UI 状态是否被正确更新 ADD 操作完成后, 搜索栏应该被关闭  
    (CLOSED)  
}
```

这一段测试的核心作用是验证 ViewModel 在执行“新增任务（ADD）”这个业务动作时，是否正确完成了两件关键事情：第一是**业务逻辑调用是否正确下沉到 Repository 层**，通过 coVerify

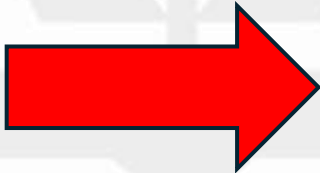
{ mockRepo.addTask(any()) } 确认确实触发了数据库新增操作；第二是**UI 状态是否同步更新**，即在执行新增后搜索栏状态被重置为 CLOSED。前面的 updateTitle 和 updateDescription 是在模拟用户已输入数据，

handleDatabaseActions (Action.ADD) 则是触发完整业务流程，这个测试本质是在验证 MVVM 中“UI事件 → ViewModel处理 → 数据层操作 + UI状态更新”这一整条链路是否正确闭环，是整个测试集中最典型的业务流程验证用例。

二、单元测试- ui.screens.list

最后，通过 JaCoCo 分析发现 ui.screens.list 包占据项目代码比例最大，同时覆盖率最低，因此后续重点针对该模块补充了大量 Compose 页面测试与交互测试。经过进一步扩展测试用例后，ui.screens.list 覆盖率提升至约 40%，项目整体测试覆盖率最终提升至 57%。整体来看，项目测试经历了从基础逻辑测试、业务逻辑 Mock 测试，到 Android UI 与 Compose 测试的逐步递进过程，最终构建出较为完整的 Android 应用测试体系。

Element	Missed Instructions	Cov.
com.ayse.todocompose.ui.screens.list		0%
com.ayse.todocompose.ui.screens.task		0%
com.ayse.todocompose.components		0%
com.ayse.todocompose.ui.theme		7%
com.ayse.todocompose.navigation.destination		0%
com.ayse.todocompose.navigation		0%
com.ayse.todocompose		0%
com.ayse.todocompose.data.repository		73%
com.ayse.todocompose.ui.viewModel		96%
com.ayse.todocompose.di		0%
com.ayse.todocompose.data		94%
com.ayse.todocompose.util		100%
com.ayse.todocompose.data.models		100%
Total	10,234 of 11,502	11%



Element	Missed Instructions	Cov.
com.ayse.todocompose.ui.screens.list		40%
com.ayse.todocompose.ui.screens.task		64%
com.ayse.todocompose.navigation.destination		0%
com.ayse.todocompose.components		83%
com.ayse.todocompose.navigation		0%
com.ayse.todocompose		0%
com.ayse.todocompose.ui.theme		83%
com.ayse.todocompose.data.repository		73%
com.ayse.todocompose.ui.viewModel		96%
com.ayse.todocompose.di		0%
com.ayse.todocompose.data		94%
com.ayse.todocompose.util		100%
com.ayse.todocompose.data.models		100%
Total	4,921 of 11,520	57%

三、集成测试

在完成单元测试与 Robolectric 测试后，项目进一步实现了基于 Android Instrumentation 的集成测试，并在 `app/src/androidTest/java/com/ayse/todocompose/integration/` 下新增了 3 个集成测试文件，共计 15 个测试用例，最终全部通过。测试主要覆盖了列表页面、任务编辑页面以及完整的应用操作流程，重点验证了 Compose UI、ViewModel 与 Room 数据库之间的数据联动是否正确。

本次集成测试采用“真实组件集成”方案，而非简单的 Composable Mock 测试。每个测试都会创建独立的内存 Room 数据库与 DataStore 文件，通过手动依赖注入构建 SharedViewModel，从而保证测试过程与真实应用运行逻辑一致。同时，测试中使用 `waitUntil()` 与 `waitForIdle()` 等方法处理 Compose 重组与 Room Flow 的异步刷新问题，并在 `@After` 中统一关闭数据库与清理临时文件，实现测试间完全隔离。

三、集成测试

测试文件	测试数量	主要验证内容
ListScreenIntegrationTest.kt	6	列表渲染、空状态、搜索、导航回调
TaskScreenIntegrationTest.kt	6	输入同步、优先级选择、Add 操作
FullAppFlowTest.kt	3	添加、删除、完整任务生命周期

Test Summary

15
tests

0
failures

0
skipped

55.885s
duration

100%
successful

Packages

Classes

Class	Tests	Failures	Skipped	Duration	Success rate
com.ayse.todocompose.integration.FullAppFlowTest	3	0	0	11.940s	100%
com.ayse.todocompose.integration.ListScreenIntegrationTest	6	0	0	22.274s	100%
com.ayse.todocompose.integration.TaskScreenIntegrationTest	6	0	0	21.671s	100%

四、性能测试-整体设计

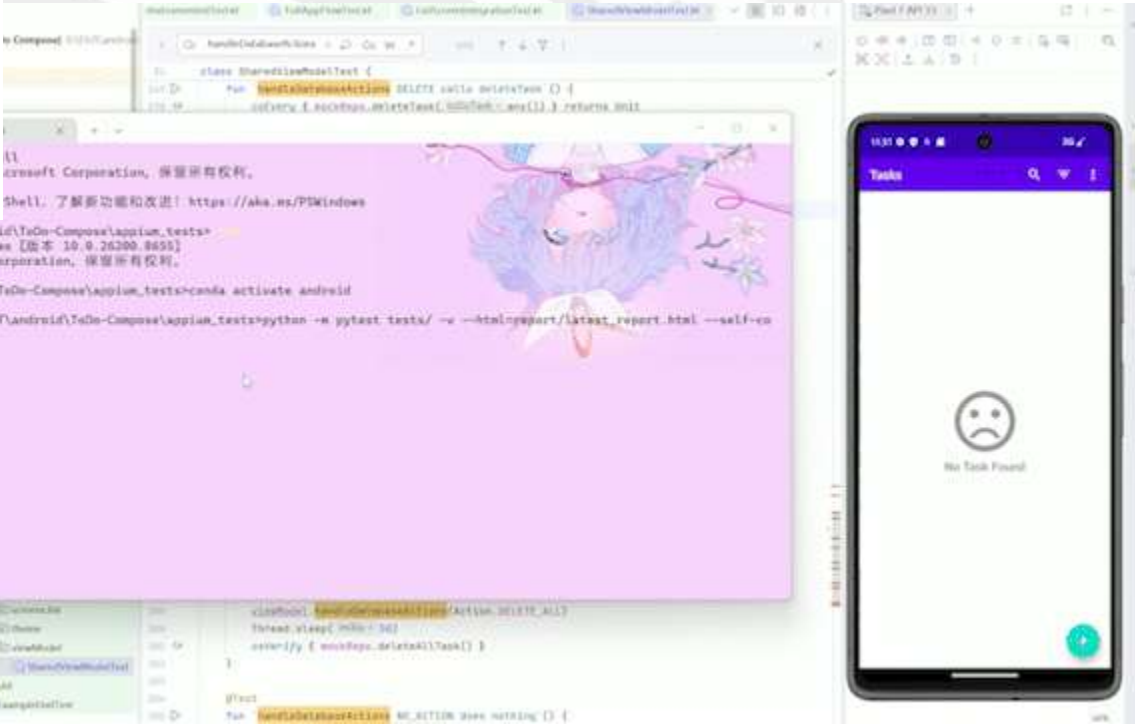
在完成单元测试、集成测试与 UI 测试后，项目进一步开展了基于 Appium 与 Pytest 的 Android 应用性能测试。性能测试主要围绕应用启动速度、任务操作响应时间、页面滚动流畅度以及内存稳定性等方面展开，共编写 13 个性能测试用例，最终全部通过。测试过程中结合 adb 性能采集工具，对启动耗时、内存 PSS、CPU 占用以及帧渲染情况进行了实时统计，并自动生成 CSV 与 HTML 格式性能报告。

整个性能测试框架位于 `appium_tests/` 目录下，采用 Python + Pytest + Appium 的方案实现。其中 `performance_collector.py` 负责通过 adb 收集 Android 性能指标，`reporter.py` 与 `gen_perf_report.py` 用于生成完整性能报告。测试运行方式为：

```
python -m pytest tests/ -v --html=report/latest_report.html --self-contained-html
```

四、性能测试-整体设计

测试类别	主要内容
启动性能测试	冷启动、热启动、启动内存与 CPU
CRUD 性能测试	添加、编辑、删除任务响应时间
页面性能测试	搜索、排序、滚动流畅度
稳定性测试	内存泄漏、空闲内存稳定性



四、性能测试-测试结果

指标	值
Cold Start (ms)	1308
Warm Start (ms)	96
Memory PSS (KB)	74772
CPU (%)	0
Frames Rendered	6
Janky Frames	3
P50 Frame Time (ms)	48
P90 Frame Time (ms)	950

问题

Frames Rendered: 6

- 总共绘制了 6 帧画面（说明测试时间很短或场景简单）

Janky Frames: 3

- 卡顿帧（掉帧 / 超时帧）
- $3/6 = 50\%$ 帧有问题

P50 Frame Time (ms): 48

- 50% 的帧渲染时间 $\leq 48\text{ms}$
- $48\text{ms} \approx$ 接近 60FPS（理想是 16ms 一帧）

P90 Frame Time (ms): 950

- 90% 的帧 $\leq 950\text{ms}$

五、总结

总体来看，本项目围绕 `ToDo-Compose` 应用构建了一套较为完整的 `Android` 测试体系，覆盖了单元测试、UI 测试、集成测试以及性能测试等多个层面。在测试过程中，逐步引入了 `JUnit`、`MockK`、`Robolectric`、`Compose Instrumentation`、`Appium` 与 `Pytest` 等技术，对应用的状态管理、数据库操作、页面交互、完整业务流程以及运行性能进行了系统性验证。尤其是针对核心类 `SharedViewModel`、`ui.screens.list` 等关键模块进行了重点测试，使项目整体测试覆盖率最终提升至 57%，核心 `ViewModel` 覆盖率达到 96%。

在此基础上，项目还进一步完成了真实 `Android` 环境下的集成与性能测试，验证了 `Room`、`Flow`、`Compose` 与 `ViewModel` 之间的数据联动能力，以及应用在长时间运行、频繁导航与列表滚动情况下的稳定性与资源占用情况。最终，全部集成测试与性能测试均顺利通过，说明该应用已经具备较好的功能正确性、稳定性与运行性能，也体现了 `Jetpack Compose + MVVM` 架构在中小型 `Android` 应用中的良好工程实践效果。